

CAP2791C: POWER BI DATA PREPARATION AND MODELING

Data Modeling Fundamentals & Dimensional Modeling

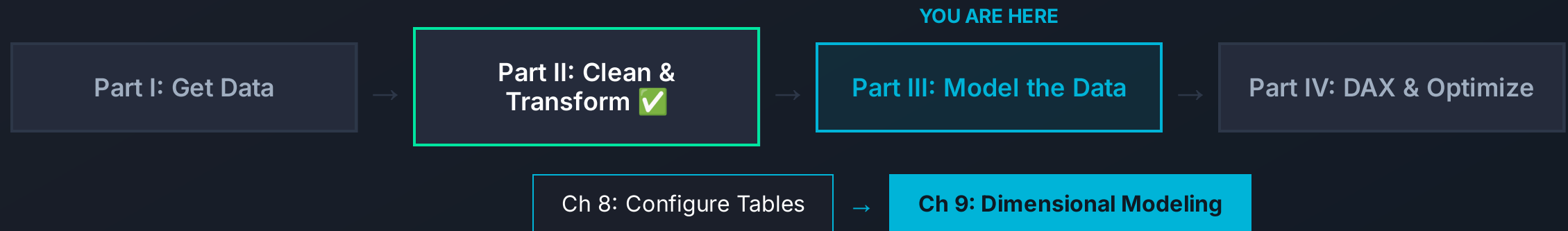
From Cleaned Tables to a Connected Star Schema

Professor Carlos Marquez

```
{ schema: "star" }
```

Where Are We in the Pipeline?

Part III begins here



✓ **Part II complete:** Your data is clean, transformed, and loaded into Power BI.

📐 **Part III focus:** Define how those isolated tables connect to each other.

- **Chapter 8:** Relationships, cardinality, and cross-filter direction.
- **Chapter 9:** Star schema, granularity, and building a Date table.

A Pile of Materials Is Not a Building

Why data modeling matters

- **Without a model:** Power BI sees 5 isolated tables — no connections, no answers.
- **With a model:** Tables are linked by shared keys → cross-table questions become possible.
- **Example question:** "Which category of product drove the most revenue at which location?" — requires 3+ tables working together.
- A **data model** is the blueprint that makes multi-table questions answerable.

WHY DOES THIS MATTER?

Every real-world business question spans multiple tables. Without a model, Power BI cannot connect the dots — and your reports will be limited to one table at a time.

WITHOUT MODEL



WITH MODEL



Every Table Has Properties Worth Configuring

Model View — the architect's drafting table

PROPERTY	WHAT IT CONTROLS	WHY IT MATTERS
 Data Type	Text, Number, Date, Currency	Ensures correct calculations and sorting
 Default Summarization	Sum, Count, Don't Summarize	Prevents meaningless ID sums in visuals
 Data Category	City, Country, Web URL, Image URL	Enables map visuals and smart defaults
 Hidden in Report View	Visible or hidden in Fields pane	Keeps report builders focused on useful columns

 PRO TIP

Set all ID columns (foreign keys) to **"Don't Summarize"** and **hide them** from Report View. They are needed for relationships — not for dragging into visuals.

Relationships Connect Tables Through Shared Keys

One shared column = one connection

- A **relationship** links two tables via a shared column (the foreign key → primary key connection).
- **Cardinality** describes how many rows match on each side:

✓ **One-to-Many (1:M)** — most common; one lookup row ↔ many transaction rows

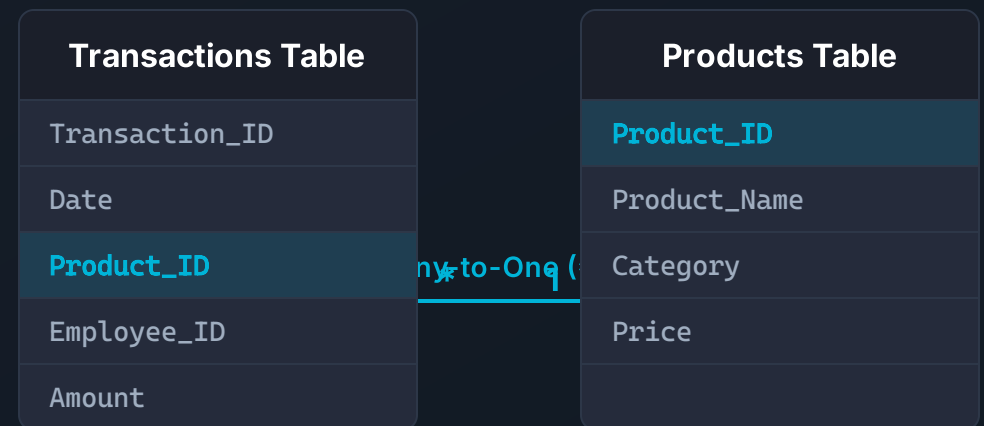
⚠ **Many-to-Many (M:M)** — use with caution; can cause unexpected results

▪ **One-to-One (1:1)** — rare; usually means the tables could be merged

FROM SQL TO POWER BI

```
FOREIGN KEY (Product_ID) REFERENCES Products(Product_ID)
```

This is the exact same concept, just built with a visual interface instead of code.



Cross-Filter Direction Controls How Filters Flow

Think of it as one-way vs. two-way streets

Single (default): Filters flow FROM the "one" side (dimension) TO the "many" side (fact)

- ✔ Selecting a product filters the transactions table
- ✔ Selecting a transaction does NOT filter the products table

Both: Filters flow in both directions

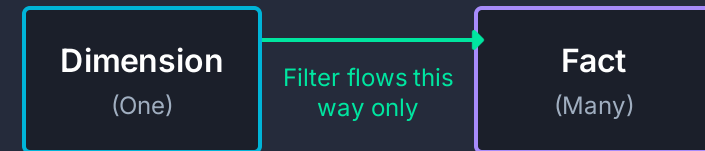
- ⚠ More flexible, but can cause unexpected results and slow performance

Rule of thumb: Default to Single; only use Both when you have a specific, tested reason.

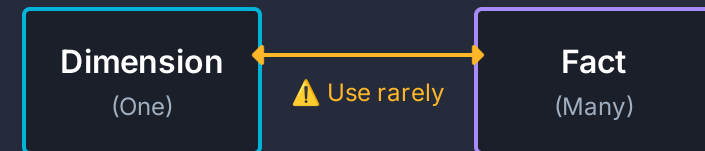
WHY?

Single-direction filtering keeps data flowing predictably — like one-way streets in a busy city grid. Bidirectional filtering works in specific cases, but defaulting to it creates confusion and performance issues.

Single (Default)



Both (Bidirectional)



Building Relationships: Drag, Drop, Confirm

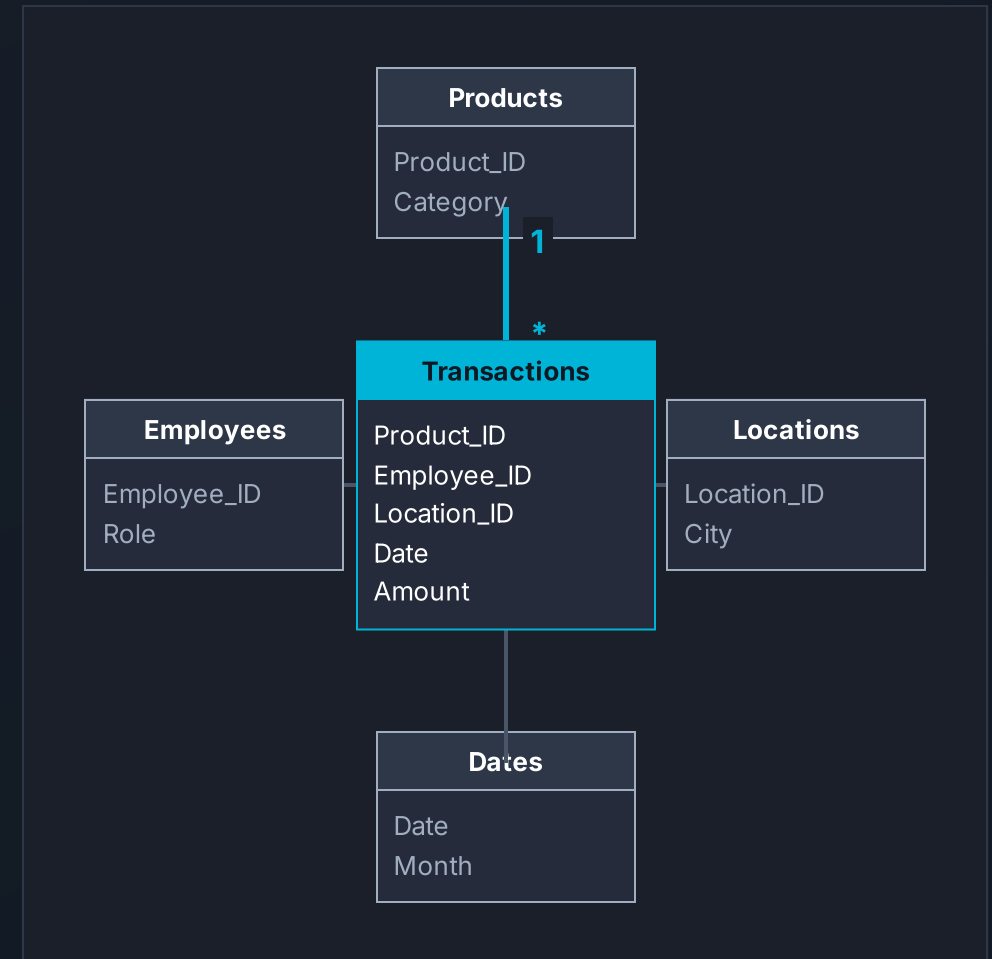
Model View walkthrough — key moments

Left sidebar → Model View icon → Drag [Foreign Key] from Fact table → Drop onto [Primary Key] in Dimension table → Confirm cardinality: Many to one (*:1) → Cross-filter: Single → OK

- Drag the foreign key column from the fact table onto the matching primary key in the dimension table.
- Confirm: cardinality = **Many to one (*:1)**, cross-filter = **Single**.

✓ HOW TO VERIFY

Switch to Report View → create a Table visual with a dimension column + a numeric measure → each group should show a different total. If every row shows the same total, the relationship is missing or misconfigured.





Part II: Dimensional Modeling

Naming the pattern you already built

Every Table Is Either a Fact or a Dimension

The most important classification in data modeling

	 FACT TABLE	 DIMENSION TABLE
Records	Events (transactions, orders, logins)	Descriptions (products, people, locations)
Row represents	One occurrence of something	One entity or category
Typical size	Large — many rows	Small — fewer rows
Contains	Numeric measures + foreign keys	Descriptive text + primary key
Example	Sales, Orders, Shipments	Products, Employees, Locations, Dates

QUICK TEST

Ask: "Does this table record events, or does it describe events?"

Events → Fact. Describes → Dimension.

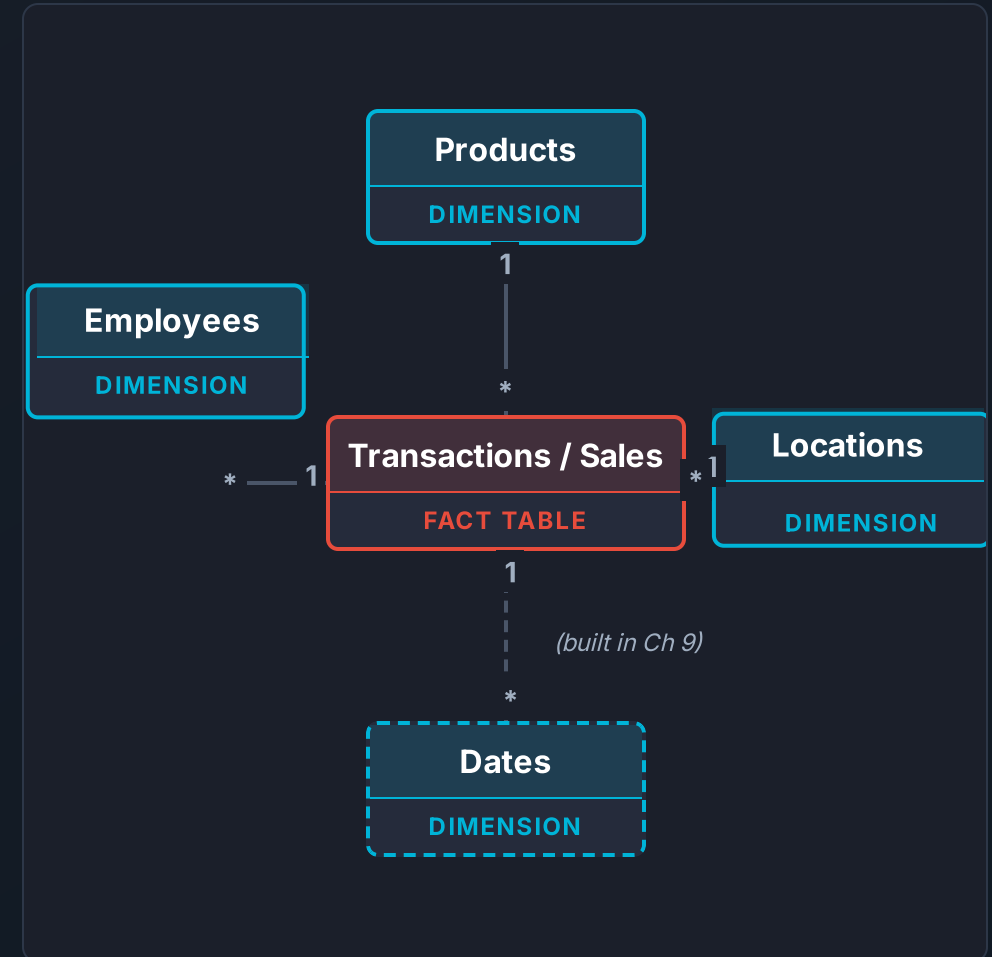
The Star Schema: One Fact, Many Dimensions

The most widely used design in business intelligence

- **Star schema:** one central fact table surrounded by dimension tables.
- **Why it works:**
 - ✓ **Readable**— the fact table is always in the center; structure is immediately clear
 - ✓ **Fast**— Power BI's VertiPaq engine is optimized for this exact pattern
 - ✓ **Predictable**— every question follows the same path: dimension → fact → measure

FROM YOUR DB COURSE

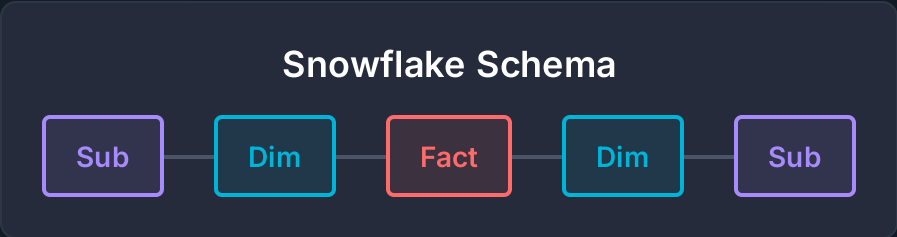
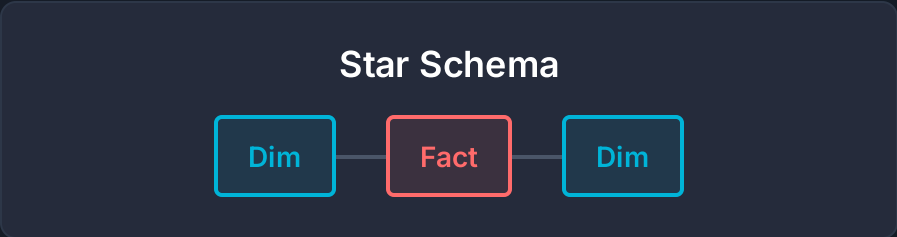
ER diagrams showed tables connected by foreign keys. A star schema is the same picture — but optimized for *reading* data (reports) rather than *writing* data (inserts/updates).



Star vs. Snowflake: Flat Dimensions vs. Normalized Dimensions

Different goals call for different designs

	STAR SCHEMA	SNOWFLAKE SCHEMA
Dimension tables	Flat — all attributes in one table	Normalized — sub-tables for sub-attributes
Query speed	✓ Faster	Slower (more joins)
Storage	Slightly more	Less redundancy
Readability	✓ Simpler	More complex
Use in Power BI	✓ Almost always	Rarely — only when forced by source



⚠ COMMON MISTAKE

Normalization was "correct" in your database course — but that was for **writing** data. For **reading** data (reports and dashboards), a flat star schema almost always wins on speed and simplicity.

Granularity: The Level of Detail in Each Row

The grain determines what questions you can answer



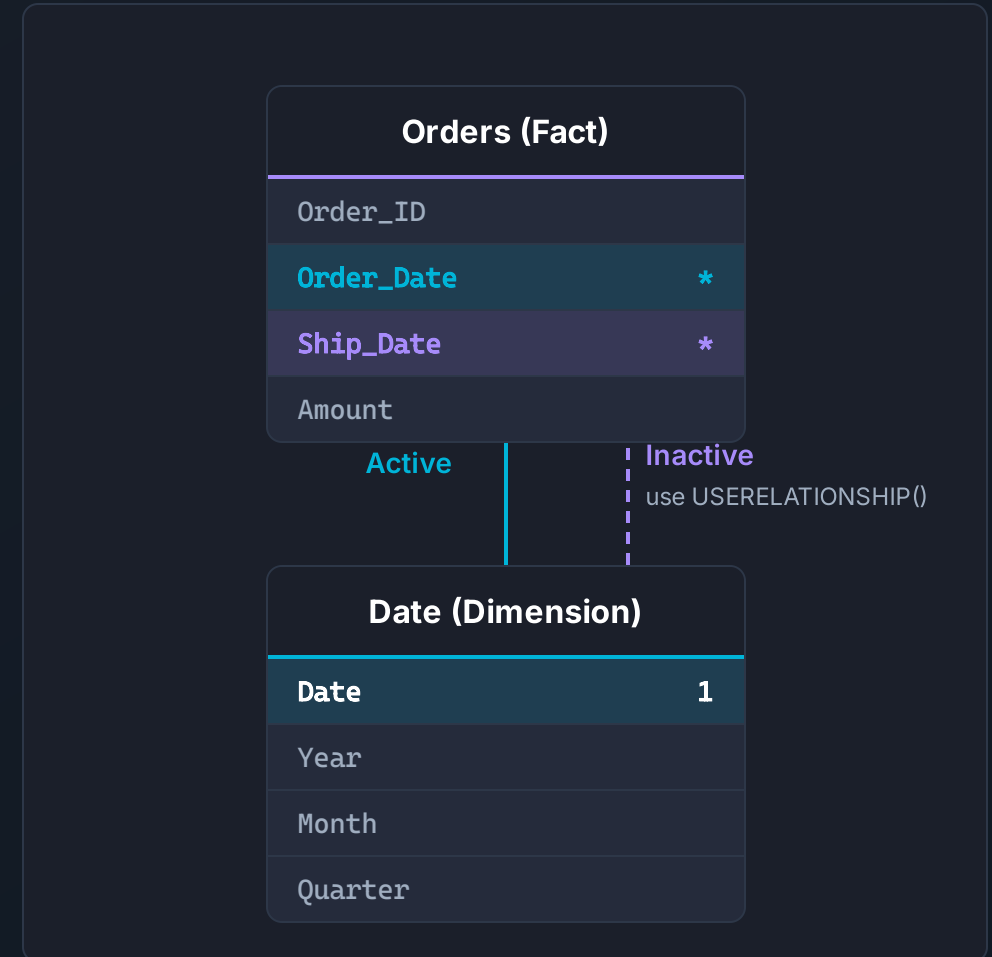
 **THE GOLDEN RULE**

Keep your fact table as granular as your source data allows. You can always summarize in a report — but you **cannot recover detail** that was aggregated away.

Role-Playing Dimensions: One Table, Two Relationships

When the same dimension serves multiple purposes

- A **role-playing dimension** is a single dimension table that connects to the fact table through more than one relationship.
- **Classic example:** a Date table connected to both an Order Date and a Ship Date.
- In Power BI: only **one relationship can be active** at a time.
- The inactive relationship is activated in DAX using `USERELATIONSHIP()` (Chapter 12).
- **Solid line** = active; **dashed line** = inactive.



The foundation for Time Intelligence

WHY BUILD ONE?



HOW TO BUILD IT

- CALENDARAUTO()



```
// Creates a single column of continuous dates  
Dates = CALENDARAUTO()
```

Date	Year	Month	Quarter
2023-01-01	2023	Jan	Q1
2023-01-02	2023	Jan	Q1
2023-01-03	2023	Jan	Q1
... continuous to end of year ...			
2023-12-31	2023	Dec	Q4

 CRUCIAL STEP

System Check: Your Data Model Is Complete

Ready for DAX and Visualization



Star Schema Applied

One central fact table surrounded by flat dimension tables.



1:M Relationships

Dimensions (1) filter Facts (*).



Single Cross-Filter

Filters flow one way, keeping performance fast and predictable.



Granularity Preserved

Fact table kept at the lowest possible level of detail.



Date Table Created

Continuous dates, marked as Date Table, connected to fact dates.



Properties Configured

ID columns hidden and set to "Don't Summarize".